

具自動化測試模型優化和即時推論資源監控之AI部署

AI Deployment with Automated Testing Model Optimization and
Real-time Inference Resource Monitoring

研究生：周暉倫

指導教授：林家瑜 博士
莊永裕 博士

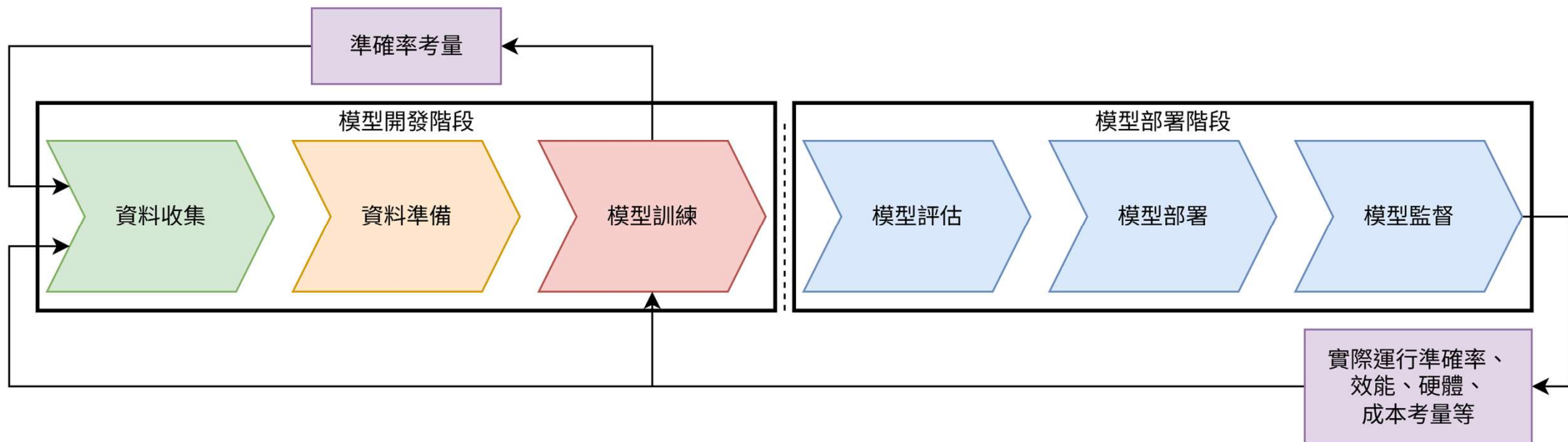


大綱

- 緒論
 - 研究背景
 - 研究動機
 - 研究目標
- 背景知識
- 系統設計
- 討論
- 結論與未來展望

研究背景

- 開發與訓練良好的機器學習模型僅僅只是機器學習生命週期的第一步，實際應用到生產環境中仍然會遇到許多的挑戰。
- 通常模型本身擁有大量參數和龐大計算量，訓練模型時只需考慮模型的準確率，進行模型部署時更需仔細考量硬體規格、資源配置以及實際效能要求等問題。



研究背景

- 目前業界為了達成上述目標，各種優化模型的技術、工具應運而生，例如 NVIDIA 提供的 TensorRT^[1]，經 TensorRT 優化後的模型可在不顯著影響推論精度的前提下，於推論時間和 GPU 記憶體用量方面均有顯著改善^[2]，而 Triton^[3]則是支援 TensorRT 等多種框架模型的部署平台。
- 優化模型後，在實際的模型部署階段為了確保模型穩定運行，監控就是重要的一環，而使用 Prometheus^[4]抓取資料配合 Grafana^[5]的儀表板呈現，是當前業界廣泛使用的開源監控解決方案。

[1] N. Corporation, "Tensorrt SDK." <https://developer.nvidia.com/tensorrt>.

[2] Y. Zhou and K. Yang, "Exploring tensorrt to improve real-time inference for deep learning", 2022.

[3] N. Corporation, "NVIDIA Triton Inference Server User Guide," 2025.

[4] Prometheus, "Prometheus: Monitoring system and time series database." <https://prometheus.io/>.

[5] Grafana Labs, "Grafana: The open and composable observability platform." <https://grafana.com/>.

研究背景

- 為了加速機器學習模型從研發到上線的流程，許多整合式平台解決方案應運而生，包括 Supervisely^[6]、DeepVis^[7]以及 MLOps^[8]工具如 Kubeflow^[9]以及 MLflow^[10]等平台。
- 這些工具或平台各有千秋：MLflow 專注於資料與模型管理；Supervisely強調部署編排與監控，但對於實際部署而言，目前仍缺少將模型部署、評估與監控等工具串聯的整合平台。

[6] Supervisely, "Supervisely: Computer vision platform for data labeling, training and deployment." <https://supervisely.com/>

[7] F. Ahmed, R. Ferdows, M. R. Islam, and A. R. M. Kamal, "Deepvis: A visual interactive system for exploring performance of deep learning models," 2022

[8] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (mlops): Overview, definition, and architecture," 2022.

[9] Kubeflow Community, "Kubeflow: Machine learning toolkit for kubernetes." <https://github.com/kubeflow/kubeflow>

[10] MLflow Authors, "MLflow: Open source platform for the machine learning lifecycle." <https://github.com/mlflow/mlflow>

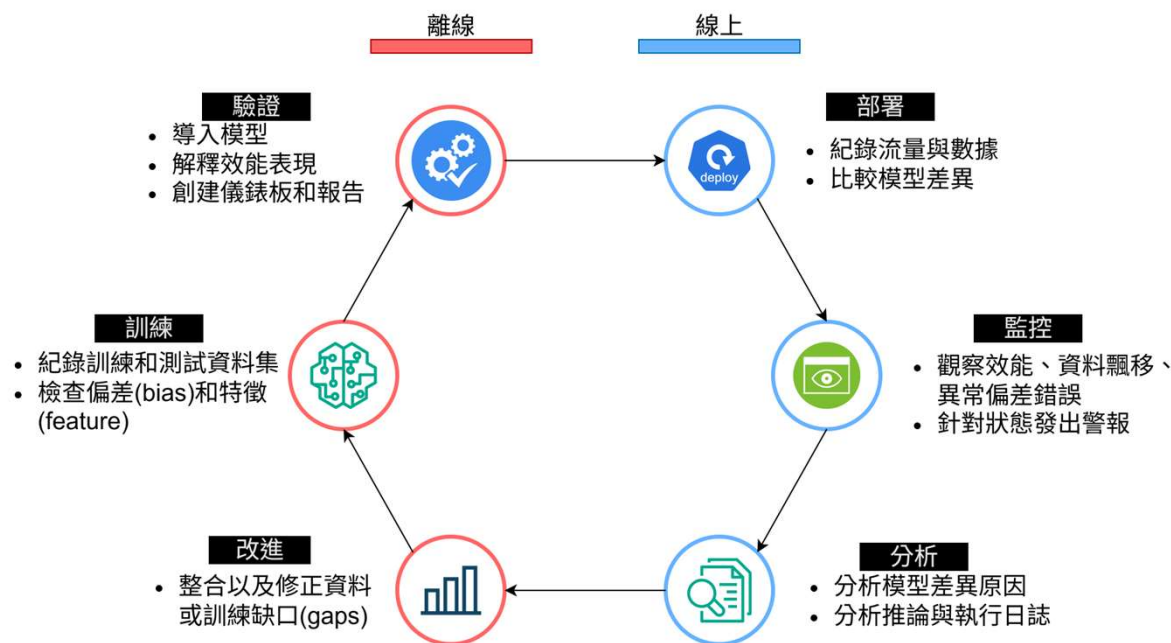
研究動機

- 部署前未考慮實際GPU、延遲等需求：
 - 模型預設通常無法滿足實際部署環境的需求，未優化的模型可能面臨如GPU記憶體使用較高、推論時間超出可接受範圍，或在特定硬體上完全無法運行等問題。
- 部署前無法預估模型優化的最佳參數：
 - 模型優化往往必須反覆試驗來尋找在特定硬體上的最佳參數組合^[11]，這種方法不僅耗時費力，還需自行整理多次測試的實驗結果，比較差異後才能找出最適合的配置。

[11] Z. Chen, Y. Cao, Y. Liu, H. Wang, T. Xie, and X. Liu, "A comprehensive study on challenges in deploying deep learning based software," 2020.

研究動機

- 部署後的效能差異及缺乏有效監控機制：
 - 在實際應用上，即使模型已通過離線測試，其在上線後仍可能因輸入分布改變、資料漂移或系統負載波動等因素，導致效能下降或預測不穩定等問題^[12]。
 - 無論是理論測試環境或實際生產部署，若缺乏有效監控，不僅會降低模型的品質與可靠度，我們也無法觀測優化的方法是否適合目前的運行狀況。



[12] A. Bodor, M. Hnida, and D. Najima, "Machine learning models monitoring in mlops context: Metrics and tools", 2022

研究目標

- 設立一個深度學習模型部署平台以降低模型部署評估的時間，並且降低人為調整參數的隨機性，最後建立從「模型優化—效能驗證—實際部署監控回饋—調整模型」的閉環。
- 系統需求
 1. 提供支援多種模型格式轉換（PyTorch → ONNX、TensorRT）的自動化模型轉換(優化)
 2. 基於量化分析的模型配置推薦功能
 - a. 可視化圖表指標分析
 - b. 指標篩選與排序功能
 - c. 二維指標權衡分析
 3. 支援 Triton 部署平台
 4. 整合 Prometheus & Grafana 硬體資源與即時推論資源監控功能
 5. 利用 Docker 進行快速容器化部署提升部署效率與擴充性



大綱

- 緒論
- 背景知識
 - 平台特點比較
- 系統設計
- 討論
- 結論與未來展望

平台特點比較

- 我們從商業領域以及先前研究中蒐集整理了一些部署平台工具
 - Supervisely
 - Lightning AI
 - Vertex AI
 - Mlflow

功能特性	Supervisely	Lightning AI	Vertex AI	MLflow	本研究平台
TensorRT整合	0	X	0	P	0
自動化模型組合轉換	X	X	X	X	0
自動化性能測試	X	X	P	X	0
部署模型推薦決策	X	X	X	X	0
視覺化決策支援	P	P	0	P	0
即時監控整合	P	P	0	X	0
開源與可客製	X	P	X	0	0

註：0完整支援、P部分支援、X不支援或有限

平台特點比較

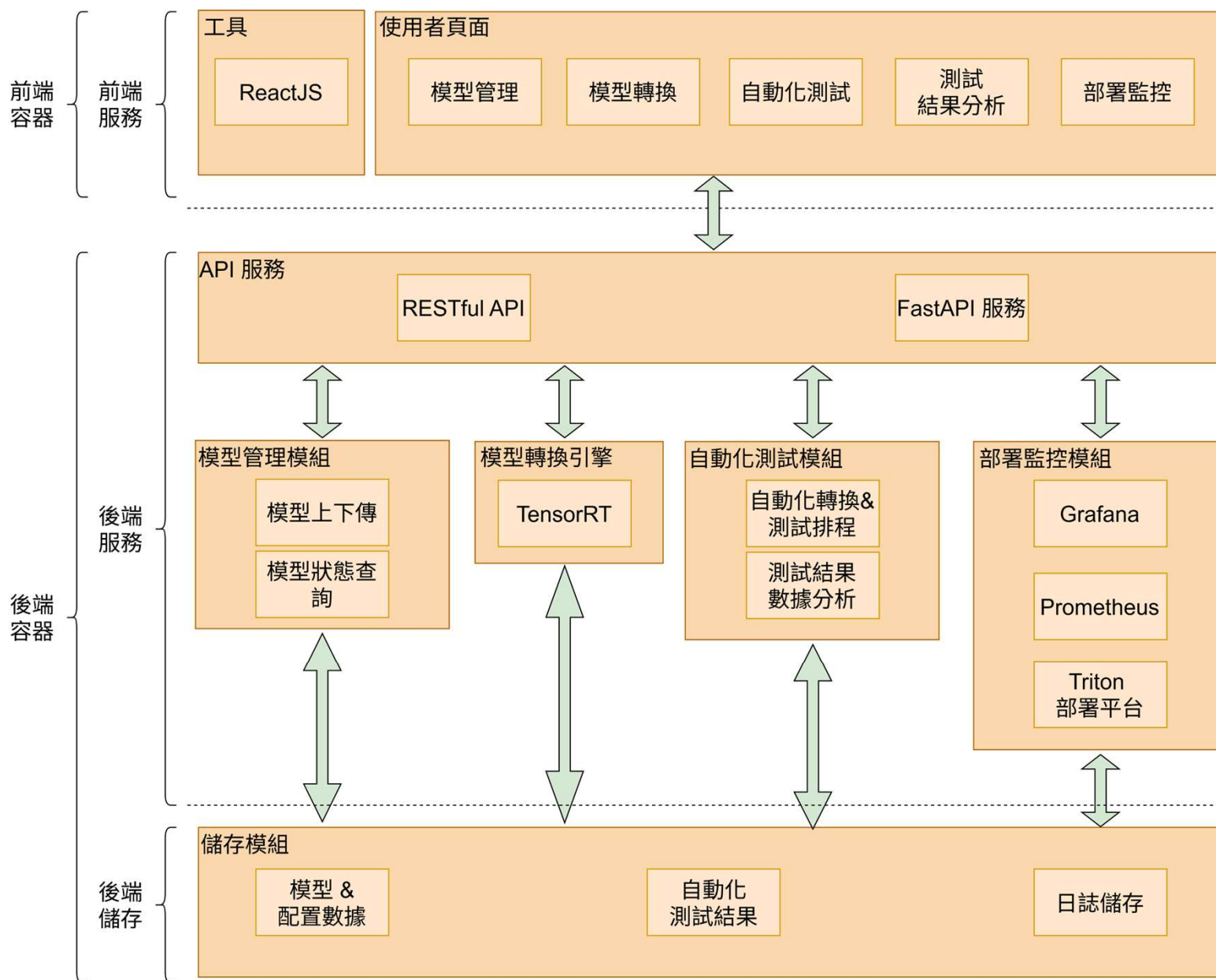
- 現有平台的限制
 - 普遍專注於模型訓練與基礎部署功能
 - 在模型優化參數選擇的自動化與性能測試整合方面存在明顯不足
 - 效能評估缺乏一個科學化流程，選擇模型的隨機性很高
 - 缺乏針對推論部署的系統化監控機制（包含資源監控與即時效能監控）



大綱

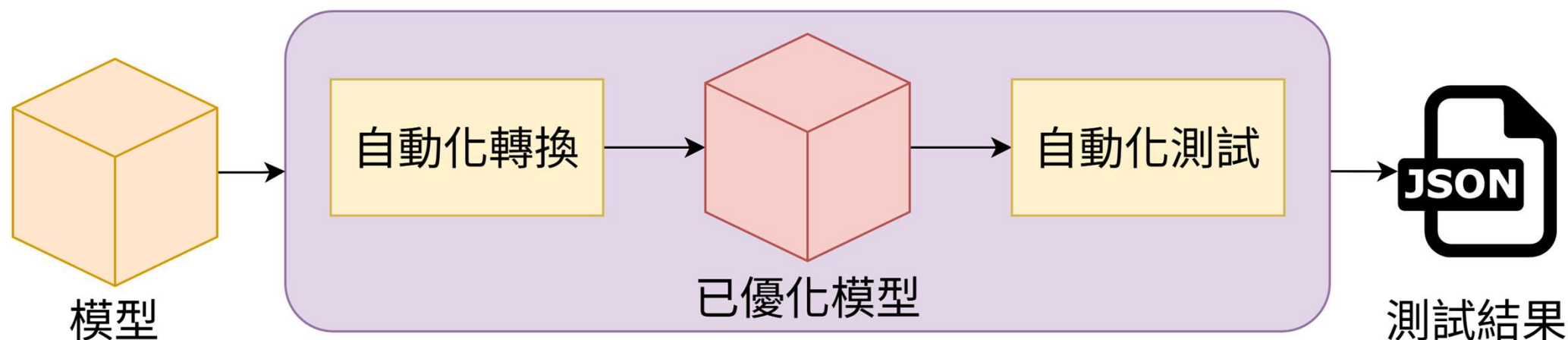
- 緒論
- 背景知識
- 系統設計
 - 系統架構
 - 自動化流程
 - 可視化圖表及部署模型選擇建議
 - 實際部署性能監控
- 討論
- 結論與未來展望

系統架構圖



自動化流程

- 將傳統上需仰賴人工腳本與多支工具串接的「模型優化—效能驗證」流程，濃縮為一條自動化資料管線。
- 減少開發者在格式轉換、效能量測與結果彙整上的重複性工作。



自動化流程 - 前端

AI部署平台

首頁

模型管理

模型優化

自動化轉換與測試

測試結果查看

自動化結果分析

部署平台監控

具自動化測試模型優化和即時推論資源監控之AI部署平台

創建自動化測試任務

* 選擇模型

test (pt)

* 模型類型 ?

物體檢測 姿態估計

* 精度選項

FP32 × FP16 ×

* 批次大小列表 (以逗號分隔) ?

1,2,4,8,16,32

* 圖像尺寸

640

* 迭代次數

100

* 驗證數據集 ?

coco coco 姿態

自定義參數 (JSON格式, 可選) ?

{'conf': 0.25, 'iou': 0.45}

開始測試

測試任務列表

任務ID	模型	創建時間	完成時間	狀態
bde1b5d...	test	2025/06/13 18:05:38	2025/06/13 19:09:42	已完成

創建自動化測試任務

* 選擇模型

test (pt)

* 模型類型 ?

物體檢測

姿態估計

* 精度選項

FP32 ×

FP16 ×

* 批次大小列表 (以逗號分隔) ?

1,2,4,8,16,32

* 圖像尺寸

640

* 迭代次數

100

* 驗證數據集 ?

coco

coco

姿態

自定義參數 (JSON格式, 可選) ?

{'conf': 0.25, 'iou': 0.45}

開始測試

已完成

推論階段: 18/18

操作

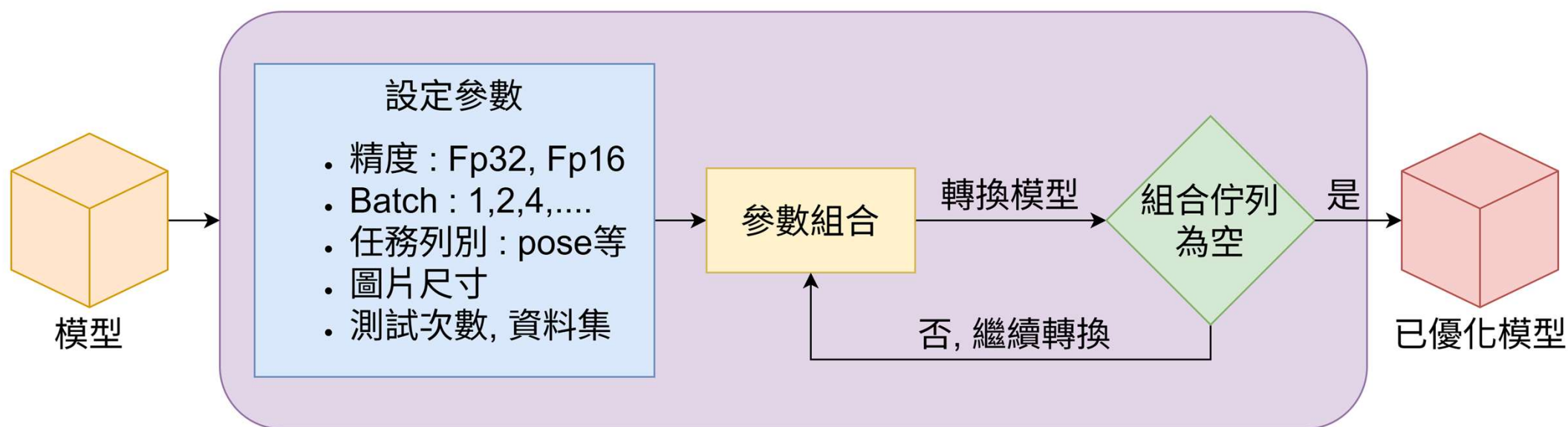
查看結果

下載完整結果

15

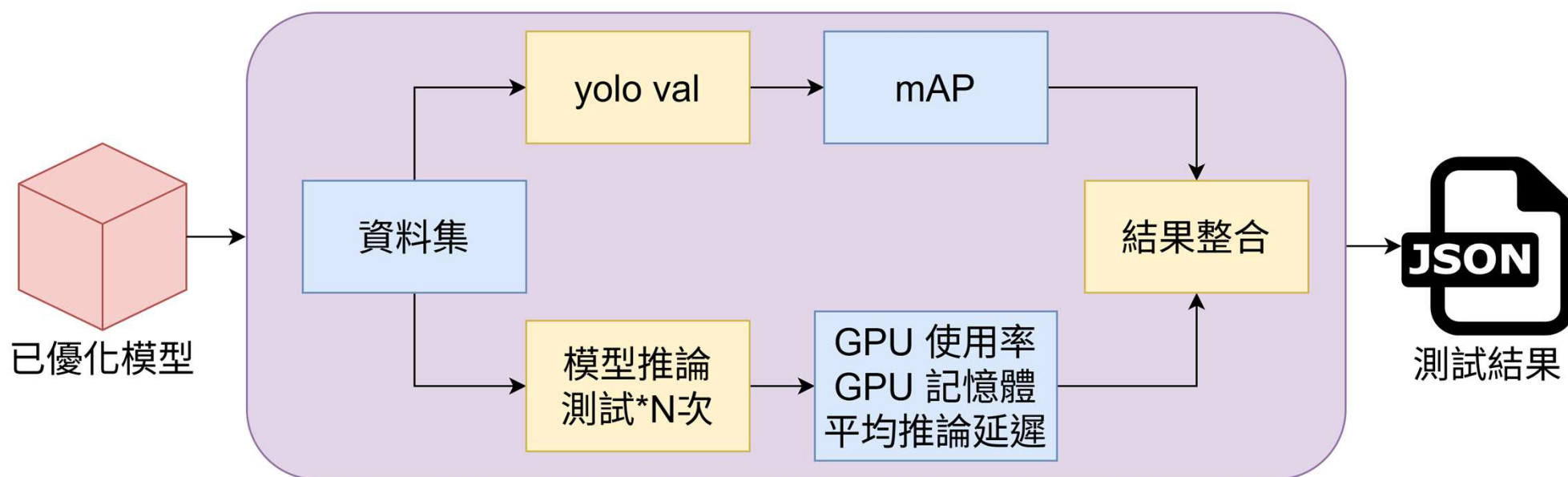
自動化流程 – 自動化轉換

- 整條「自動化轉換」管線以模型原始權重作為起點，當使用者將模型權重上傳至平台並設定完轉換任務後，系統會轉換所有的參數組合。



自動化流程 – 自動化測試

- 完成 TensorRT 佇列轉換後，每一組優化引擎將自動排入測試佇列等待評測，目標是在無人工介入的情況下，對所有優化模型一次性收集「**精度，效能指標**」與「**硬體指標**」。



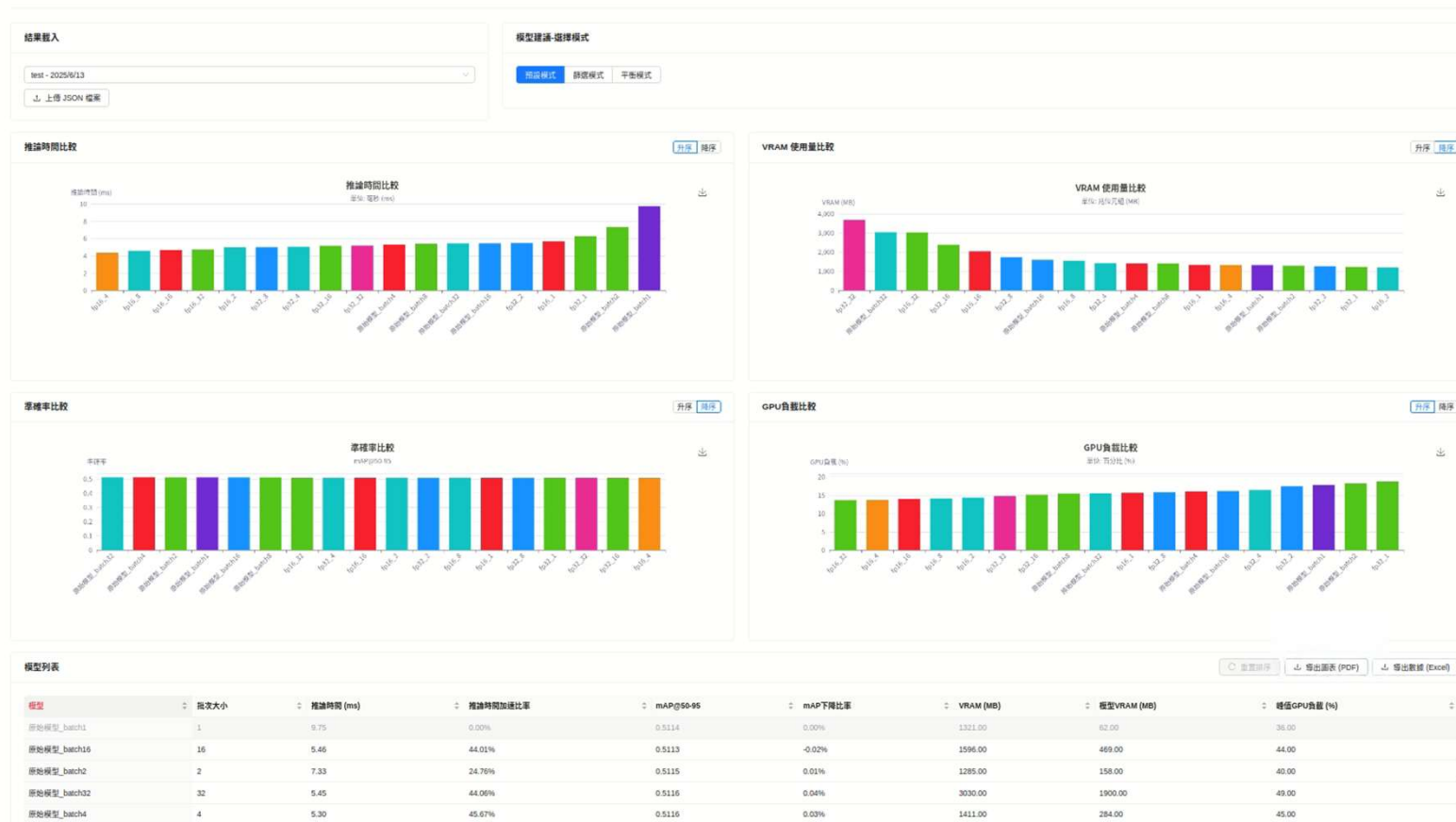
可視化圖表及部署模型選擇建議

- 自動化流程完成後，系統產生了包含所有測試結果的結構化 JSON 檔案，但使用者面臨一個關鍵問題：如何從數十種模型配置中選擇最適合的部署方案？
- 傳統方法需要使用者逐一檢視每個模型的測試結果，手動比較不同配置的效能表現，這不僅耗時費力，更容易因為數據量龐大而遺漏重要資訊。
- 解決方案：
 - 可視化圖表：將複雜的效能數據轉化為直觀的圖表呈現
 - 多模式選擇：提供篩選模式和平衡模式等滿足不同需求
 - 推薦機制：基於量化分析自動提供最佳配置建議

可視化圖表 - 手動排序模式

- 分析儀表板採用視覺化設計圖表呈現，提供使用者手動排序結果與建議目標，使研究人員能夠快速識別不同模型配置間的效能差距。

自動化測試結果分析儀表板



可視化圖表 – 篩選模式

- 篩選模式採用多維度條件篩選機制，允許使用者根據實際部署環境的硬體限制與效能需求設定篩選條件，系統將即時篩選出符合所有條件的模型配置。

模型建議-選擇模式

預設模式 **篩選模式** 平衡模式

☒ 是否加入原始模型(僅調整batch)比較 ⌂ 重置篩選條件

推論時間上限: 10

10

ms

VRAM上限: 8192

8192

MB

GPU負載上限: 90

90

%

mAP下限: 0.4

0.4

模型VRAM上限: 2048

2048

MB

模型建議方式 (選擇指標作為排序目標):

以推論時間為目標 (越低越好) 以mAP為目標 (越高越好) 以VRAM為目標 (越低越好) 以模型VRAM為目標 (越低越好) 以GPU負載為目標 (越低越好)

可視化圖表 – 平衡(權衡)模式

- 平衡模式透過評估準確率下降比率與推論時間加速比率兩者之間的取捨，透過標準化二維散點圖和幾何距離演算法，提供權衡的模型選擇建議。
- 方法：
 - Min-Max 標準化處理：將所有指標映射至 $[0, 1]$ 標準化座標空間
 - $x_{norm} = \frac{x_{speedup} - x_{min}}{x_{max} - x_{min}}$ ，推論時間加速比率標準化值（值越高表示速度提升越顯著）
 - $y_{norm} = \frac{y_{speedup} - y_{min}}{y_{max} - y_{min}}$ ，準確率下降比率標準化值（值越高表示精度損失越大）
 - 幾何距離計算：測量各模型配置至理想平衡線（ $y=x$ 對角線）的垂直距離
 - $d = \frac{|x_{norm} - y_{norm}|}{\sqrt{2}}$
 - 極值點設定：避免任一模型成為標準化極值，允許使用者設定指標權重，極值點的指標倍率設定越大表示使用者較看重該指標。

可視化圖表 - 平衡(權衡)模式

模型建議-選擇模式

預設模式 篩選模式 平衡模式

☐ 是否加入原始模型(僅調整batch)比較 ☒ 重置篩選條件

推論時間上限: 10

VRAM上限: 8192

GPU負載上限: 90

1 ms

8 MB

90 %

mAP下限: 0.4

模型VRAM上限: 2048

0.4

2 MB

極值參考點設置

推論時間加速比的倍率(相較MAX值): 1.5

準確率下降比的倍率(相較MAX值): 1.2

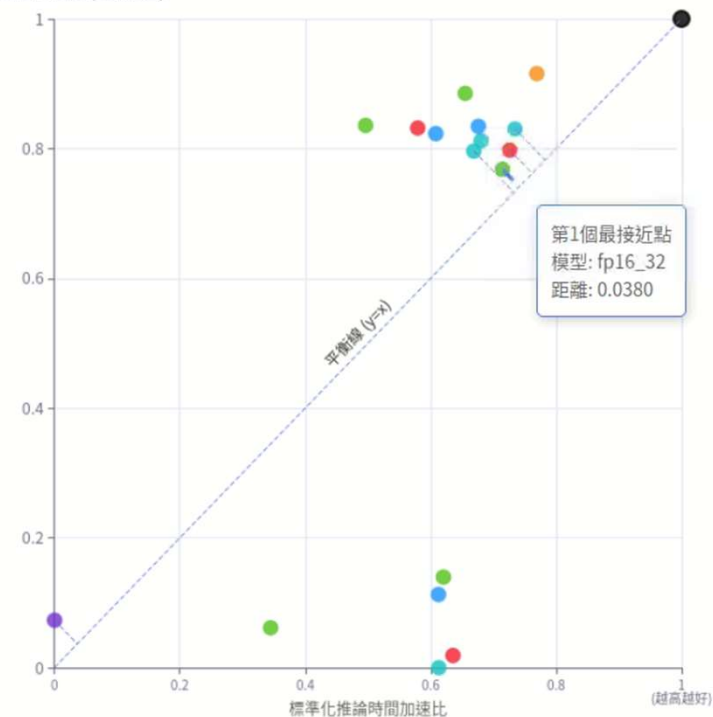
1.5

1.2

平衡模式分析 - 推論時間加速比 vs 準確率下降比

標準化準確率下降比 (越高越差)

最靠近對角線(y=x)的為最佳平衡



實際部署性能監控 - 系統資源監控

- 當研究人員透過自動化測試平台獲得模型建議並完成初步選擇後，系統進入實際部署階段的持續性能監控流程，核心目標在於驗證測試環境下的理論性能是否能在真實運行環境中實現。
- 系統資源監控著重於硬體層級的性能指標追蹤，採用 Prometheus 作為數據收集引擎，結合 Grafana 儀表板進行即時視覺化呈現，以時間序列的方式記錄所有性能數據。



實際部署性能監控 – 部署平台監控

- 部署平台監控聚焦於模型層級的運行狀態與推論性能，系統提供整合式監控介面，即時呈現所有已部署模型的詳細資訊，當監控數據顯示當前部署的模型未能達到預期性能水準時，也能夠輕易切換其餘模型。

部署平台監控

[刷新](#)

Triton服務器狀態

🟢 運行正常

已掛載模型數量

📊 3

總推論次數

📊 300

模型格式分佈

1 PT 1 ENGINE 1 ONNX

已掛載模型列表

數據每30秒自動刷新 • 最後更新: 下午2:05:55

模型名稱	格式	精度	批次大小	狀態	推論次數	總延遲	推理延遲	最後推論	操作
test test	PT	預設	批次: 1	● READY	100	472.67 ms	21.75 ms	2分鐘前	🔗 卸載
test test_91460434_engine_fp32_batch1_size640	ENGINE	FP32	批次: 1	● READY	100	407.50 ms	2.23 ms	1分鐘前	🔗 卸載
test test_91460434_onnx_fp32_batch1_size640	ONNX	FP32	批次: 1	● READY	100	400.05 ms	11.49 ms	剛剛	🔗 卸載

第 1-3 項，共 3 項 < 1 > 10 條/頁



大綱

- 緒論
- 背景知識
- 系統設計
- 討論
 - 情境討論
- 結論與未來展望

情境討論

- 為了驗證本研究所提出的自動化模型優化與推薦系統的實際效益，我們設計了三種典型的實際應用情境進行深入分析：
 - 案例一：高負載情境下的推論時間優化
 - 案例二：資源受限環境下的優化
 - 案例三：多目標平衡(權衡)的綜合優化
- NVIDIA RTX 3090 GPU、64GB DDR4 記憶體、Intel Core i9-10900K
- YOLOv8n - Pose
- COCO 2017 - Pose
- 100 次的重複測試推論效能
- 推論延遲、VRAM使用量、GPU使用率(%)以及mAP@0.5:0.95精度分數
- 參數組合：精度(FP32, FP16), 批次(1, 2, 4, 8, 16, 32)
- 比較對象：原始模型 vs 預設參數轉換模型(FP32, 1) vs 建議配置轉換模型

案例一：高負載情境下的推論時間優化

- 在高負載的生產環境中，推論時間是決定系統可用性的關鍵因素，本案例目標是在保持可接受精度水準的前提下，實現最低的推論時間。
- 在這情況下，我們將採用手動排序模式，直接排序推論時間獲得推薦模型。

建議部署模型列表

[↓ 導出圖表 \(PDF\)](#)
[↓ 導出數據 \(Excel\)](#)

模型	批次大小	推論時間 (ms)	推論時間加速比率	mAP@50-95	mAP下降比率	VRAM (MB)	模型VRAM (MB)	峰值GPU負載 (%)
fp16_4	4	4.36	55.29%	0.5092	-0.43%	1322.00	206.00	35.00
fp16_8	8	4.60	52.80%	0.5095	-0.39%	1550.00	434.00	35.00
fp16_16	16	4.66	52.19%	0.5095	-0.37%	2037.00	921.00	35.00
fp16_32	32	4.74	51.35%	0.5096	-0.35%	3017.00	1874.00	36.00
fp16_2	2	4.98	48.88%	0.5095	-0.38%	1202.00	86.00	34.00
fp32_8	8	5.01	48.60%	0.5094	-0.39%	1734.00	618.00	41.00
fp32_4	4	5.06	48.07%	0.5095	-0.37%	1424.00	308.00	41.00
fp32_16	16	5.16	47.07%	0.5093	-0.41%	2372.00	1256.00	41.00
原始模型_batch4	4	5.30	45.67%	0.5116	0.03%	1411.00	284.00	45.00
原始模型_batch8	8	5.40	44.57%	0.5113	-0.03%	1411.00	284.00	44.00

案例一：高負載情境下的推論時間優化 - 結果比較

- 系統建議配置(fp16_4)在降低延遲時間方面表現卓越，相較於原始模型實現了**55.29%** 的推論時間降低，也比預設參數優化的模型還要更加快速。

模型配置	推論延遲 (ms)	延遲改善 (%)	mAP@0.5:0.95	精度下降 (%)
原始訓練模型	9.75	-	0.5114	-
預設參數優化 (fp32_1)	6.27	35.65	0.5094	0.39
系統建議配置 (fp16_4)	4.36	55.29	0.5092	0.43

案例二：資源受限環境下的優化

- 在邊緣設備或資源受限的部署環境中，VRAM使用量往往是決定模型可部署性的關鍵，本案例將在 GPU 限制下追求最低推論時間，將使用篩選模式來取得建議。
- 約束條件：
 - 總 VRAM 使用量上限：1,300MB
 - 模型 VRAM 佔用上限：150MB
 - GPU 負載上限：90%

模型建議-選擇模式

☐ 預設模式
 ☒ 篩選模式
 ☐ 平衡模式

☒ 是否加入原始模型(僅調整batch)比較
 ☐ 重置篩選條件

推論時間上限: 10 ms
 VRAM上限: 1300 MB
 GPU負載上限: 90 %

mAP下限: 0.4
 模型VRAM上限: 150 MB

模型建議方式 (選擇指標作為排序目標):

☒ 以推論時間為目標 (越低越好)
 ☐ 以mAP為目標 (越高越好)
 ☐ 以VRAM為目標 (越低越好)
 ☐ 以模型VRAM為目標 (越低越好)
 ☐ 以GPU負載為目標 (越低越好)

建議部署模型列表

模型	批次大小	推論時間 (ms)	推論時間加速比率	mAP@50-95	mAP下降比率	VRAM (MB)	模型VRAM (MB)	峰值GPU負載 (%)
fp16_2	2	4.98	48.88%	0.5095	-0.38%	1202.00	86.00	34.00
fp32_2	2	5.49	43.72%	0.5095	-0.38%	1266.00	150.00	41.00
fp32_1	1	6.27	35.65%	0.5094	-0.39%	1226.00	96.00	41.00

案例二：資源受限環境下的優化 - 結果比較

- 原始訓練模型由於未經壓縮與優化，超出 GPU 記憶體限制而無法部署
- 預設參數模型能夠降低 GPU 記憶體使用量得以部署，但推論時間並非最快
- 系統建議配置(fp16_2)在推論時間優先模式下表現卓越，相較於原始模型實現了 48.88% 的推論時間降低，同時滿足所有 VRAM 約束條件。

模型配置	推論延遲 (ms)	延遲改善 (%)	總 VRAM(MB)	模型 VRAM(MB)
原始訓練模型	9.75	-	1321(X)	127
預設參數優化 (fp32_1)	6.27	35.65	1226	96
系統建議配置 (fp16_2)	4.98	48.88	1202	86

註：X 超出限制

案例三：多目標平衡(權衡)的綜合優化

- 在實際應用中，需要在多個性能指標間尋求最佳平衡點，既不能犧牲過多精度，也不能容忍過高的推論時間。本案例專注於尋找**精度下降與推論時間加速比率最接近理想平衡線**的配置，將**採用平衡模式來取得建議**。
- 條件：
 - 極值參考點倍率參數：
 - 推論時間加速比倍率設為1.5
 - 準確率下降比倍率設為1

模型建議-選擇模式

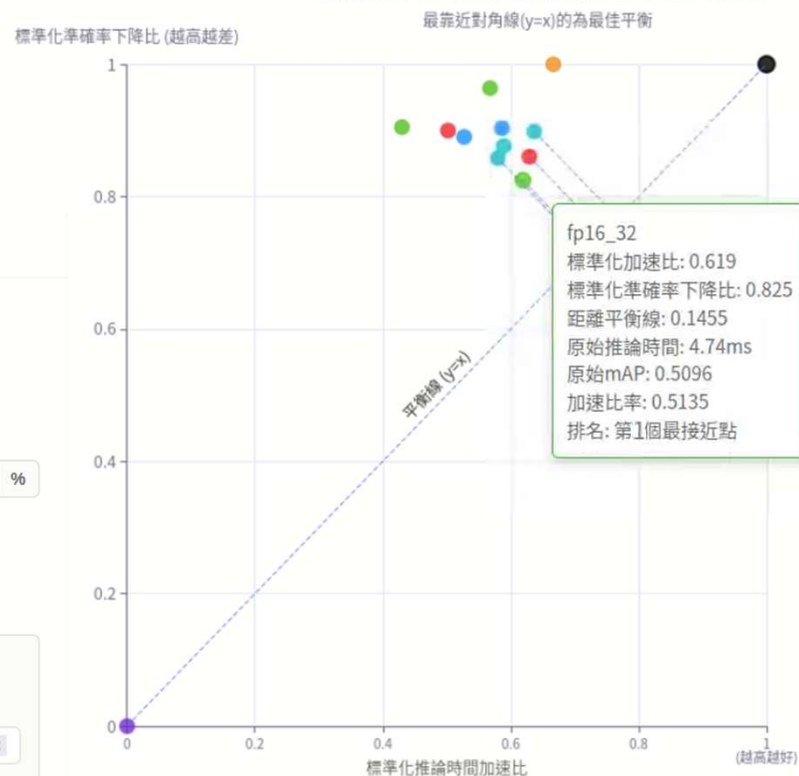
☒ 是否加入原始模型(僅調整batch)比較
 ☐ 重置篩選條件

推論時間上限: 10 ms
 VRAM上限: 8192 MB
 GPU負載上限: 90 %

mAP 下限: 0.4
 模型VRAM上限: 2048 MB

極值參考點設置
 推論時間加速比的倍率(相較MAX值): 1.5
 準確率下降比的倍率(相較MAX值): 1

平衡模式分析 - 推論時間加速比 vs 準確率下降比



案例三：多目標平衡(權衡)的綜合優化

- 原始模型與極值參考點不列入推薦排行

建議部署模型列表

📄 導出圖表 (PDF)

📄 導出數據 (Excel)

模型	批次大小	推論時間 (ms)	推論時間加速比率	mAP@50-95	mAP下降比率	VRAM (MB)	模型VRAM (MB)	峰值GPU負載 (%)	與y=x軸的距離
原始模型_batch1	1	9.75	0.00%	0.5114	0.00%	1321.00	62.00	36.00	0.0000
極值參考點	0	0.00	82.93%	0.0000	-0.43%	0.00	0.00	0.00	0.0000
fp16_32	32	4.74	51.35%	0.5096	-0.35%	3017.00	1874.00	36.00	0.1455
fp16_16	16	4.66	52.19%	0.5095	-0.37%	2037.00	921.00	35.00	0.1631
fp16_8	8	4.60	52.80%	0.5095	-0.39%	1550.00	434.00	35.00	0.1852
fp32_4	4	5.06	48.07%	0.5095	-0.37%	1424.00	308.00	41.00	0.1972
fp16_2	2	4.98	48.88%	0.5095	-0.38%	1202.00	86.00	34.00	0.2032
fp32_8	8	5.01	48.60%	0.5094	-0.39%	1734.00	618.00	41.00	0.2244
fp16_4	4	4.36	55.29%	0.5092	-0.43%	1322.00	206.00	35.00	0.2357
fp32_2	2	5.49	43.72%	0.5095	-0.38%	1266.00	150.00	41.00	0.2565

案例三：多目標平衡(權衡)的綜合優化 - 結果比較

- 系統建議配置 (fp16_32) 在多目標平衡分析中表現最優，其標準化準確率下降比為0.825以及標準化推論時間加速比為0.619
 - 距離理想平衡線只有 $d = \frac{|0.619-0.825|}{\sqrt{2}} = 0.1455$
 - 實現了 51.35% 的推論時間改善，同時精度下降控制在 0.35% 以內。

模型配置	推論延遲 (ms)	延遲改善 (%)	mAP@0.5:0.95	精度下降 (%)	平衡距離
原始訓練模型	9.75	-	0.5114	-	-
預設參數優化 (fp32_1)	6.27	35.65	0.5094	0.39	0.3360
系統建議配置 (fp16_32)	4.74	51.35	0.5096	0.35	0.1455

案例討論 – 總結

- 案例一（高負載情境下的推論時間優化）：系統建議配置實現 55.29% 推論時間降低，證實智慧化推薦系統能有效識別高負載情境核心需求
- 案例二（資源受限環境下的優化）：在嚴格 GPU 約束下實現 48.88% 推論時間改善，展現系統在資源受限環境的優化建議有效性
- 案例三（多目標平衡(權衡)的綜合優化）：最優平衡距離 0.1455，有 51.35% 推論時間改善，驗證多目標優化演算法能夠在權衡模型時提供有效的建議
- 通過三個典型案例的深入分析，驗證了本研究所提出的自動化優化測試與推薦系統能在各個情境下提供不同的適配建議



大綱

- 緒論
- 背景知識
- 系統設計
- 討論
- 結論與未來展望

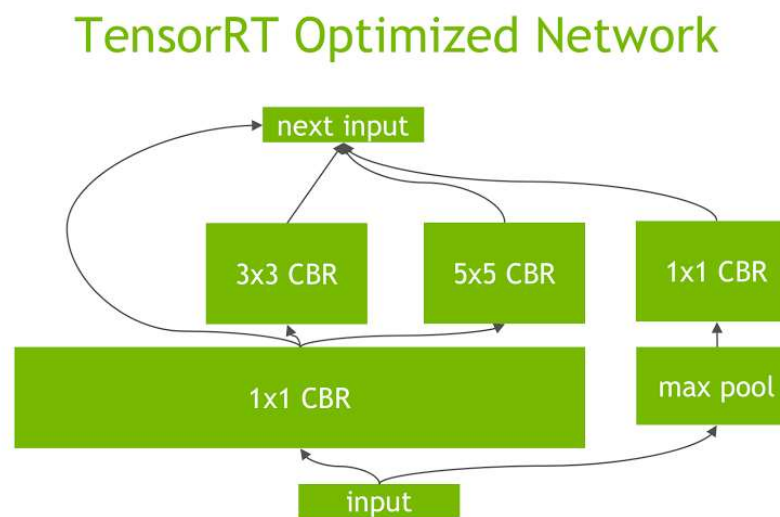
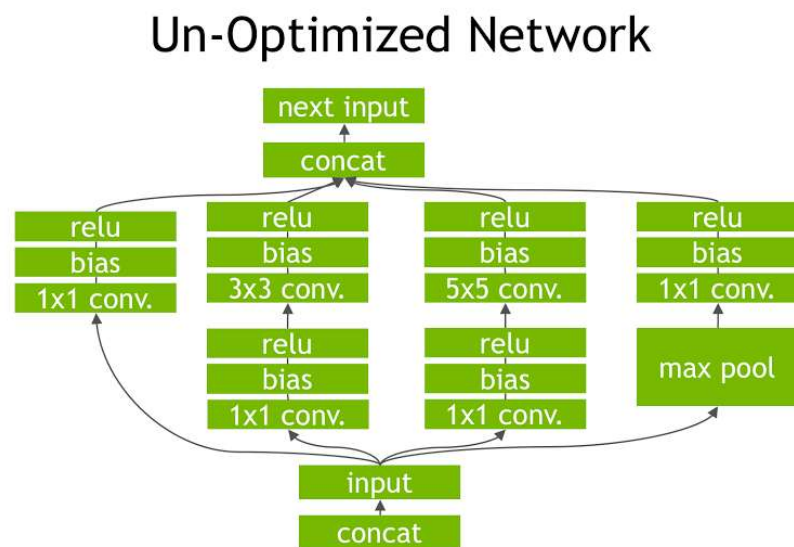
結論與未來展望

- 核心貢獻
 - 自動化流程以及可視化決策支援，可以大幅提升優化及部署效率，讓缺乏優化經驗的研究人員也能快速獲得專業的部署建議，降低深度學習模型部署的技術門檻。
 - 監控與回饋機制：建立雙層監控架構（硬體資源監控 + 推論性能監控），並形成完整的測試、驗證與調整循環機制，可以大幅提升系統可靠性與運維效率。
- 未來展望
 - 跨平台硬體支援擴展
 - Intel OpenVINO等優化引擎，提供真正的跨平台模型優化能力
 - 模型架構支援擴展
 - 支援其他種類模型，開發針對不同架構的專用優化策略
 - 智慧化推薦改進
 - 引入機器學習技術改進推薦演算法，結合強化學習技術實現自適應優化能力
 - 開發完整的可解釋性機制，讓使用者理解「為什麼這是最佳配置」
 - 端到端的完全自動化
 - 探索從原始資料到部署就緒模型的全程自動化，結合 CI/CD, AutoML 技術實現聯合優化

感謝各位的聆聽

TensorRT

- TensorRT^[1] 是 NVIDIA 推出的高效能深度學習推論引擎，旨在協助使用者針對 NVIDIA GPU 硬體特性進行模型部署優化並大幅提升推理效能，主要使用 layer/Tensor fusion 以及 低精度量化 等技術來優化模型效能。



[1] N. Corporation, "Tensorrt SDK." <https://developer.nvidia.com/tensorrt>.
 圖片來源：<https://developer.nvidia.com/blog/tensorrt-3-faster-tensorflow-inference/>

Triton Inference Server

- Triton Inference Server 是 NVIDIA 開源的多框架推論服務平台，旨在簡化 AI 模型在生產環境的部署並提升效能，與 TensorRT 具有深度整合優勢。
- 特點：
 - 支援 TensorFlow, PyTorch, ONNX, TensorRT 等多框架模型
 - 內建模型推論佇列等機制以提高吞吐量
 - 提供 RestFul & Grpc 的推論方案，適合低延遲需求場景

Prometheus & Grafana

- Prometheus 與 Grafana 是當今主流的開源監控解決方案組合，主要能夠幫助使用者提供可靠的監控與紀錄功能。
- 特點：
 - Prometheus 透過主動拉取的方式，定期收集指標資料並以時間戳組成時序資料儲存
 - Grafana 提供視覺化工具展示 Prometheus 所抓取的監控數據
 - 支援歷史趨勢圖及即時更新



Docker

- Docker 是一個開源的容器化平台，透過輕量級虛擬化技術讓應用程式及其相依套件打包成可移植的容器映像檔。
- 特點：
 - 確保在不同環境中執行的一致性，解決「在我的電腦上可以運行」的問題
 - 針對部署的系統可加快其部署效率
 - 適用於微服務架構、雲端部署、開發環境標準化等場景